Reviewer Pack — Minimal Rank of Algebraic Curves over Function Fields vs Mon…

Entry 88163488fc07 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 14:11:49 UTC

Minimal Rank of Algebraic Curves over Function Fields vs Monotone Circuit Depth for k-CLIQUE

Entry ID: `88163488fc07` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 14:11:49 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Algebraic Curves) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

> ['For every function field F of characteristic zero, there exists a constant c > 0 such that for all monomially equivalent algebraic curves C and D over F, the monotone circuit depth of C is at most c times the monotone circuit depth of D.', 'The monotone circuit depth of an algebraic curve C over F is defined as the minimum depth of a monotone circuit computing the incidence relation on C.', 'For any instance of k-CLIQUE with n vertices, the monotone circuit depth of its characteristic polynomial is O(n^(1/4)).']

Rationale (proposer's reasoning):

> ['Algebraic geometry provides rich invariants that could potentially explain lower bounds in complexity theory. The rank of an algebraic curve over a function field is a well-studied invariant, and if this rank can be shown to relate to the circuit depth of a monotone circuit, it might expose new insights into the complexity of k-CLIQUE.', 'The conjecture proposes that the structure of the algebraic curves over function fields, captured by their ranks, could serve as a bridge between algebraic geometry and computational complexity.']

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ec02e5c7d52836d6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each pair of monomially equivalent algebraic curves C and D over function field F, if the monotone circuit depth of C (depth_C) is less than or equal to a constant c times the monotone circuit depth of D (depth_D), then the conjecture is supported. Conversely, if any curve's monotone circuit depth exceeds c times its rank by more than 3 for any seed, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `algebraic geometry algebraic curves AND monotone circuit complexity` - `monomially equivalent algebraic curves over function fields AND monotone circuit depth` - `incidence relation on algebraic curves AND k-CLIQUE characteristic polynomial monotone circuit depth O(n^(1/4))`

Top relevant hits considered: - [http://arxiv.org/abs/0912.2255v3] Test ideals via algebras of p^{-e}-linear maps - [http://arxiv.org/abs/1812.11710v1] Geometric Satake correspondence for affine Kac-Moody Lie algebras of type A - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/1409.0951v2] Arithmetic geometry of algebraic curves and their moduli space - [http://arxiv.org/abs/0904.2058v1] The Power of Depth 2 Circuits over Algebras - [http://arxiv.org/abs/2210.04450v2] Height moduli on cyclotomic stacks and counting elliptic curves over function fields - [http://arxiv.org/abs/1710.05836v2] Estimating the Contribution of Dynamical Ejecta in the Kilonova Associated with GW170817 - [http://arxiv.org/abs/2404.17623v3] Broadband Multi-wavelength Properties of M87 during the 2018 EHT Campaign including a Very High Energy Flaring Episode

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define constants and parameters
    n = 20  # Number of vertices in k-CLIQUE instance
    c = 3   # Constant to be estimated

    # Generate a random instance of k-CLIQUE
    graph = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    # Compute the characteristic polynomial of the graph
    def char_poly(poly):
        if len(poly) == 1:
            return poly[0]
        else:
```

```
            return [poly[-1]] + [x * poly[i] - poly[-2] * poly[i-1] for i in range(1, len(poly))]

    char_poly = [1] + [-sum(graph[i][j] for j in range(n)) if i == 0 else sum(graph[i][j] for j in ra

    # Compute the monotone circuit depth of the characteristic polynomial
    def circuit_depth(poly):
        if len(poly) == 1:
            return 1
        else:
            return max(circuit_depth([poly[-1]] + [x * poly[i] - poly[-2] * poly[i-1] for i in ran

    depth = circuit_depth(char_poly)

    # Estimate the constant c by comparing the rank and monotone circuit depth
    rank = sum(graph[i][j] for i in range(n) for j in range(i+1))

    if rank == 0:
        return {
            "metric_name": "c",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    c_estimated = depth / rank

    # Check if the conjecture holds for this instance
    conjecture_holds = c_estimated <= c

    return {
        "metric_name": "c",
        "metric_value": c_estimated,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"c_estimated={c_estimated} > c={c}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(res["conjecture_holds"] for res in results):
        mean_c = sum(res["metric_value"] for res in results) / len(results)
        support_fraction = 1.0
        result = f"RESULT: SUPPORTED mean={mean_c} std=0 support_fraction={support_fraction}"
    else:
        first_failing_seed = next((res["seed"] for res in results if not res["conjecture_holds"]),
        counterexample = next((res["counterexample"] for res in results if not res["conjecture_hol
        result = f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first
```

```
    print(result)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ed15707f.py", line 77, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ed15707f.py", line 44, in run_trial
    depth = circuit_depth(char_poly)
            ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ed15707f.py", line 42, in circuit_depth
      return max(circuit_depth([poly[-1]] + [x * poly[i] - poly[-2] * poly[i-
1] for i in range(1, len(poly))]), circuit_depth(poly[:-1])) + 1
                                                ^
NameError: name 'x' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14656
2	propose	ollama_remote	glm4:latest	0	0	14533
3	propose	ollama_remote	glm4:latest	0	0	14193
4	preregistration	ollama_remote	glm4:latest	0	0	9679
5	novelty	ollama_remote	glm4:latest	0	0	10220
6	novelty	ollama_remote	glm4:latest	0	0	10419
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17339
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12284
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15719
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8969

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
11	judge	ollama_remote	glm4:latest	0	0	13054

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 141065 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/88163488fc07.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/88163488fc07.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/88163488fc07.tar.gz` (if generated)